In Kotlin, by default, all variables are marked as 'not null' unless you explicitly tell the compiler that the variable can be null with a question mark (?) operator. This means that in order to declare a null variable, you need to add '?' at the end of the variable type.

Even if you try to set a non-null variable to null, the compiler will 'yell' at you.

```
// Compile time error
var spell: String = "Lumos"
spell = null

// OK
var spell: String? = "Lumos"
spell = null
```

## Using null variables

Directly accessing null variables results in compile time error, unless protection has been cast before using these variables. If the null is safely handled then the compiler is smart enough to know that the variable (the 'spell' variable, in our case) won't show an error on using it further in the *if* block.

```
// Compile time error
val spell: String? = null
val length = spell.length

// OK
val spell: String? = null
if (spell != null) {
  // Smart cast
  val length = spell.length
}
```

The idiomatic way is as follows:

```
// Safe
val spell: String? = null
val length = spell?.length

// Safe with else
val length = spell?.length ?: -1
```

In the first case, if the value for *spell* exists then the *length* will be returned, else *null* will be returned without any exception. A point to note over here is the 'val length' will be a nullable value, which means the compiler will force you to check for *null* value in case of the 'length' variable.

The second case is pretty straightforward. Unlike the first case, the 'val length' won't be nullable over here because we have handled it using the *else* condition.

## String interpolation

String interpolation is a nifty feature, which makes your code cleaner. It makes it easier to build long strings.

```
fun sayHello(message: String) {
   println("Welcome $message")
}

fun sayTime() {
   println("Time: ${System.currentTimeMillis()}")
}
```

Using *$* allows you to avoid String concat or StringBuilder. You can execute a small piece of code using *${}*. Internally, it uses StringBuilder for performance on the JVM level.

## Lambda expressions

Lambda expressions are anonymous functions, which can be passed as arguments to other functions. For people who are not aware about Lambda expressions, I recommend they read more about them.

```
// Java
mButton.setOnClickListener(new OnClickListener() {
        public void onClick(View v) {
            // Your logic
        }
});

// Kotlin
mButton.setOnClickListener {
   // Your logic
}

mButton.setOnClickListener {
   // Using the view parameter
   v -> v.visibility = GONE
}
```

## Higher order functions

Kotlin supports Higher order functions which means functions can be passed to another function as parameters.

An example is given below:

```
fun consume(f: () -> Unit): Boolean {
   f()
   return true
}
```

The ugly part of receiving functions as arguments is that the compiler needs to create classes for them, which can impact performance. But in Kotlin, this can be easily solved by using the keyword *inline*. An inline function will have less impact on performance, because it will substitute the call to the function with its code in compilation time. So it won't